

¡Por supuesto! Te ayudaré a integrar un sistema completo de control de stock en tiempo real. Primero necesito revisar la estructura actual del proyecto para adaptar la solución. Perfecto, te ayudaré a integrar un sistema completo de control de stock en tiempo real. Voy a crear una solución integral que incluya:

Sistema de Control de Stock en Tiempo Real

Características que implementaremos:

1.  Campo de cantidad/stock en productos
2.  Actualización automática al generar facturas
3.  Alertas de stock bajo
4.  Historial de movimientos
5.  Dashboard de inventario
6.  Indicadores visuales de stock

PASO 1: Actualizar Base de Datos

Primero, añadiremos las tablas y campos necesarios:

```
-- Agregar campos de stock a la tabla productos (si no existen)
ALTER TABLE products
ADD COLUMN stock INT DEFAULT 0,
ADD COLUMN min_stock INT DEFAULT 10,
ADD COLUMN stock_status ENUM('in_stock', 'low_stock', 'out_of_stock') DEFAULT 'in_stock';
```

```
-- Crear tabla para historial de movimientos de stock
CREATE TABLE IF NOT EXISTS stock_movements (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  product_id BIGINT NOT NULL,
  movement_type ENUM('IN', 'OUT', 'ADJUSTMENT') NOT NULL,
  quantity INT NOT NULL,
  previous_stock INT NOT NULL,
  new_stock INT NOT NULL,
  reference_type VARCHAR(50), -- 'INVOICE', 'PURCHASE', 'ADJUSTMENT'
  reference_id BIGINT,
  reason VARCHAR(255),
  created_by VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
```

```
);
```

```
-- Índices para mejorar performance
```

```
CREATE INDEX idx_stock_movements_product ON stock_movements(product_id);
```

```
CREATE INDEX idx_stock_movements_date ON stock_movements(created_at);
```

```
CREATE INDEX idx_products_stock_status ON products(stock_status);
```

PASO 2: Backend - Entidades Java

1. Actualizar entidad Product

```
// ... existing code ...
```

```
@Entity
```

```
@Table(name = "products")
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    // ... existing fields ...
```

```
    @Column(name = "stock", nullable = false)
```

```
    private Integer stock = 0;
```

```
    @Column(name = "min_stock", nullable = false)
```

```
    private Integer minStock = 10;
```

```
    @Enumerated(EnumType.STRING)
```

```
    @Column(name = "stock_status")
```

```
    private StockStatus stockStatus = StockStatus.IN_STOCK;
```

```
    // Getters y Setters
```

```
    public Integer getStock() {
```

```
        return stock;
```

```
    }
```

```
    public void setStock(Integer stock) {
```

```
        this.stock = stock;
```

```
        updateStockStatus();
```

```
    }
```

```
    public Integer getMinStock() {
```

```

        return minStock;
    }

    public void setMinStock(Integer minStock) {
        this.minStock = minStock;
        updateStockStatus();
    }

    public StockStatus getStockStatus() {
        return stockStatus;
    }

    public void setStockStatus(StockStatus stockStatus) {
        this.stockStatus = stockStatus;
    }

    // Método para actualizar estado automáticamente
    private void updateStockStatus() {
        if (stock <= 0) {
            this.stockStatus = StockStatus.OUT_OF_STOCK;
        } else if (stock <= minStock) {
            this.stockStatus = StockStatus.LOW_STOCK;
        } else {
            this.stockStatus = StockStatus.IN_STOCK;
        }
    }

    // Método helper para reducir stock
    public boolean reduceStock(Integer quantity) {
        if (this.stock >= quantity) {
            this.stock -= quantity;
            updateStockStatus();
            return true;
        }
        return false;
    }

    // Método helper para aumentar stock
    public void increaseStock(Integer quantity) {
        this.stock += quantity;
        updateStockStatus();
    }
}

```

2. Crear enum StockStatus

```
package in.bushansirgur.springboot.entity;
```

```
public enum StockStatus {  
    IN_STOCK("En Stock"),  
    LOW_STOCK("Stock Bajo"),  
    OUT_OF_STOCK("Sin Stock");  
  
    private final String displayName;  
  
    StockStatus(String displayName) {  
        this.displayName = displayName;  
    }  
  
    public String getDisplayName() {  
        return displayName;  
    }  
}
```

3. Crear entidad StockMovement

```
package in.bushansirgur.springboot.entity;
```

```
import javax.persistence.*;  
import java.time.LocalDateTime;
```

```
@Entity  
@Table(name = "stock_movements")  
public class StockMovement {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @ManyToOne(fetch = FetchType.LAZY)  
    @JoinColumn(name = "product_id", nullable = false)  
    private Product product;  
  
    @Enumerated(EnumType.STRING)  
    @Column(name = "movement_type", nullable = false)  
    private MovementType movementType;  
  
    @Column(nullable = false)
```

```

private Integer quantity;

@Column(name = "previous_stock", nullable = false)
private Integer previousStock;

@Column(name = "new_stock", nullable = false)
private Integer newStock;

@Column(name = "reference_type")
private String referenceType;

@Column(name = "reference_id")
private Long referenceld;

private String reason;

@Column(name = "created_by")
private String createdBy;

@Column(name = "created_at")
private LocalDateTime createdAt = LocalDateTime.now();

// Constructores
public StockMovement() {}

public StockMovement(Product product, MovementType movementType, Integer quantity,
    Integer previousStock, Integer newStock, String referenceType,
    Long referenceld, String reason, String createdBy) {
    this.product = product;
    this.movementType = movementType;
    this.quantity = quantity;
    this.previousStock = previousStock;
    this.newStock = newStock;
    this.referenceType = referenceType;
    this.referenceld = referenceld;
    this.reason = reason;
    this.createdBy = createdBy;
    this.createdAt = LocalDateTime.now();
}

// Getters y Setters
public Long getId() {
    return id;
}

```

```
public void setId(Long id) {  
    this.id = id;  
}
```

```
public Product getProduct() {  
    return product;  
}
```

```
public void setProduct(Product product) {  
    this.product = product;  
}
```

```
public MovementType getMovementType() {  
    return movementType;  
}
```

```
public void setMovementType(MovementType movementType) {  
    this.movementType = movementType;  
}
```

```
public Integer getQuantity() {  
    return quantity;  
}
```

```
public void setQuantity(Integer quantity) {  
    this.quantity = quantity;  
}
```

```
public Integer getPreviousStock() {  
    return previousStock;  
}
```

```
public void setPreviousStock(Integer previousStock) {  
    this.previousStock = previousStock;  
}
```

```
public Integer getNewStock() {  
    return newStock;  
}
```

```
public void setNewStock(Integer newStock) {  
    this.newStock = newStock;  
}
```

```

    public String getReferenceType() {
        return referenceType;
    }

    public void setReferenceType(String referenceType) {
        this.referenceType = referenceType;
    }

    public Long getReferenceld() {
        return referenceld;
    }

    public void setReferenceld(Long referenceld) {
        this.referenceld = referenceld;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }

    public String getCreatedBy() {
        return createdBy;
    }

    public void setCreatedBy(String createdBy) {
        this.createdBy = createdBy;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }
}

```

4. Crear enum MovementType

```
package in.bushansirgur.springboot.entity;
```

```
public enum MovementType {  
    IN("Entrada"),  
    OUT("Salida"),  
    ADJUSTMENT("Ajuste");  
  
    private final String displayName;  
  
    MovementType(String displayName) {  
        this.displayName = displayName;  
    }  
  
    public String getDisplayName() {  
        return displayName;  
    }  
}
```

PASO 3: Backend - Repositorios

Repository para StockMovement

```
package in.bushansirgur.springboot.repository;
```

```
import in.bushansirgur.springboot.entity.StockMovement;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.data.jpa.repository.Query;  
import org.springframework.stereotype.Repository;
```

```
import java.time.LocalDateTime;  
import java.util.List;
```

```
@Repository
```

```
public interface StockMovementRepository extends JpaRepository<StockMovement, Long> {
```

```
    // Obtener movimientos por producto
```

```
    List<StockMovement> findByProductIdOrderByCreatedAtDesc(Long productId);
```

```
    // Obtener movimientos recientes
```

```
    List<StockMovement> findTop50ByOrderByCreatedAtDesc();
```

```
    // Movimientos por rango de fecha
```

```
    List<StockMovement> findByCreatedAtBetweenOrderByCreatedAtDesc(
```



```

        LocalDateTime start, LocalDateTime end);

    // Movimientos por tipo
    List<StockMovement> findByMovementTypeOrderByCreatedAtDesc(String movementType);
}

```

Actualizar ProductRepository

```

// ... existing code ...

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // ... existing methods ...

    // Productos con stock bajo
    @Query("SELECT p FROM Product p WHERE p.stock <= p.minStock AND p.stock > 0")
    List<Product> findLowStockProducts();

    // Productos sin stock
    @Query("SELECT p FROM Product p WHERE p.stock = 0")
    List<Product> findOutOfStockProducts();

    // Productos por estado de stock
    List<Product> findByStockStatus(StockStatus stockStatus);

    // Contar productos por estado
    Long countByStockStatus(StockStatus stockStatus);
}

```

PASO 4: Backend - Servicio de Stock

```

package in.bushansirgur.springboot.service;

import in.bushansirgur.springboot.entity.*;
import in.bushansirgur.springboot.repository.ProductRepository;
import in.bushansirgur.springboot.repository.StockMovementRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

```

```

import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Service
public class StockService {

    @Autowired
    private ProductRepository productRepository;

    @Autowired
    private StockMovementRepository stockMovementRepository;

    /**
     * Reduce el stock de un producto (usado en ventas)
     */
    @Transactional
    public boolean reduceStock(Long productId, Integer quantity, String referenceType,
                               Long referenceId, String userName) {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new RuntimeException("Producto no encontrado"));

        Integer previousStock = product.getStock();

        if (!product.reduceStock(quantity)) {
            return false; // No hay suficiente stock
        }

        productRepository.save(product);

        // Registrar movimiento
        StockMovement movement = new StockMovement(
            product,
            MovementType.OUT,
            quantity,
            previousStock,
            product.getStock(),
            referenceType,
            referenceId,
            "Venta realizada",
            userName
        );

        stockMovementRepository.save(movement);
    }
}

```

```

        return true;
    }

    /**
     * Aumenta el stock de un producto (usado en compras/entradas)
     */
    @Transactional
    public void increaseStock(Long productId, Integer quantity, String reason, String userName)
    {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new RuntimeException("Producto no encontrado"));

        Integer previousStock = product.getStock();
        product.increaseStock(quantity);
        productRepository.save(product);

        // Registrar movimiento
        StockMovement movement = new StockMovement(
            product,
            MovementType.IN,
            quantity,
            previousStock,
            product.getStock(),
            "MANUAL",
            null,
            reason,
            userName
        );

        stockMovementRepository.save(movement);
    }

    /**
     * Ajuste manual de stock
     */
    @Transactional
    public void adjustStock(Long productId, Integer newStock, String reason, String userName)
    {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new RuntimeException("Producto no encontrado"));

        Integer previousStock = product.getStock();
        Integer difference = newStock - previousStock;
    }

```

```

    product.setStock(newStock);
    productRepository.save(product);

    // Registrar movimiento
    StockMovement movement = new StockMovement(
        product,
        MovementType.ADJUSTMENT,
        Math.abs(difference),
        previousStock,
        newStock,
        "ADJUSTMENT",
        null,
        reason,
        userName
    );

    stockMovementRepository.save(movement);
}

/**
 * Obtener historial de movimientos de un producto
 */
public List<StockMovement> getProductMovements(Long productId) {
    return stockMovementRepository.findByProductIdOrderByCreatedAtDesc(productId);
}

/**
 * Obtener movimientos recientes
 */
public List<StockMovement> getRecentMovements() {
    return stockMovementRepository.findTop50ByOrderByCreatedAtDesc();
}

/**
 * Obtener productos con stock bajo
 */
public List<Product> getLowStockProducts() {
    return productRepository.findLowStockProducts();
}

/**
 * Obtener productos sin stock
 */

```

```

    public List<Product> getOutOfStockProducts() {
        return productRepository.findOutOfStockProducts();
    }

    /**
     * Dashboard de estadísticas de stock
     */
    public Map<String, Object> getStockDashboard() {
        Map<String, Object> dashboard = new HashMap<>();

        dashboard.put("totalProducts", productRepository.count());
        dashboard.put("inStock",
productRepository.countByStockStatus(StockStatus.IN_STOCK));
        dashboard.put("lowStock",
productRepository.countByStockStatus(StockStatus.LOW_STOCK));
        dashboard.put("outOfStock",
productRepository.countByStockStatus(StockStatus.OUT_OF_STOCK));
        dashboard.put("lowStockProducts", getLowStockProducts());
        dashboard.put("outOfStockProducts", getOutOfStockProducts());
        dashboard.put("recentMovements", getRecentMovements());

        return dashboard;
    }

    /**
     * Validar si hay stock suficiente para una venta
     */
    public boolean hasEnoughStock(Long productId, Integer quantity) {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new RuntimeException("Producto no encontrado"));
        return product.getStock() >= quantity;
    }
}

```

PASO 5: Backend - Controlador de Stock

```

package in.bushansirgur.springboot.controller;

import in.bushansirgur.springboot.entity.Product;
import in.bushansirgur.springboot.entity.StockMovement;
import in.bushansirgur.springboot.service.StockService;
import org.springframework.beans.factory.annotation.Autowired;

```

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/v1/stock")
@CrossOrigin(origins = "http://localhost:5173")
public class StockController {

    @Autowired
    private StockService stockService;

    /**
     * Aumentar stock (entrada de mercancía)
     */
    @PostMapping("/increase/{productId}")
    public ResponseEntity<Map<String, Object>> increaseStock(
        @PathVariable Long productId,
        @RequestParam Integer quantity,
        @RequestParam String reason,
        @RequestParam(required = false, defaultValue = "System") String userName) {

        try {
            stockService.increaseStock(productId, quantity, reason, userName);

            Map<String, Object> response = new HashMap<>();
            response.put("success", true);
            response.put("message", "Stock aumentado correctamente");

            return ResponseEntity.ok(response);
        } catch (Exception e) {
            Map<String, Object> response = new HashMap<>();
            response.put("success", false);
            response.put("message", "Error al aumentar stock: " + e.getMessage());

            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
        }
    }

    /**

```

```

* Ajustar stock manualmente
*/
@PutMapping("/adjust/{productId}")
public ResponseEntity<Map<String, Object>> adjustStock(
    @PathVariable Long productId,
    @RequestParam Integer newStock,
    @RequestParam String reason,
    @RequestParam(required = false, defaultValue = "System") String userName) {

    try {
        stockService.adjustStock(productId, newStock, reason, userName);

        Map<String, Object> response = new HashMap<>();
        response.put("success", true);
        response.put("message", "Stock ajustado correctamente");

        return ResponseEntity.ok(response);
    } catch (Exception e) {
        Map<String, Object> response = new HashMap<>();
        response.put("success", false);
        response.put("message", "Error al ajustar stock: " + e.getMessage());

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
    }
}

/**
 * Obtener historial de movimientos de un producto
 */
@GetMapping("/movements/{productId}")
public ResponseEntity<List<StockMovement>> getProductMovements(@PathVariable Long
productId) {
    List<StockMovement> movements = stockService.getProductMovements(productId);
    return ResponseEntity.ok(movements);
}

/**
 * Obtener movimientos recientes
 */
@GetMapping("/movements/recent")
public ResponseEntity<List<StockMovement>> getRecentMovements() {
    List<StockMovement> movements = stockService.getRecentMovements();
    return ResponseEntity.ok(movements);
}

```

```

/**
 * Obtener productos con stock bajo
 */
@GetMapping("/low-stock")
public ResponseEntity<List<Product>> getLowStockProducts() {
    List<Product> products = stockService.getLowStockProducts();
    return ResponseEntity.ok(products);
}

/**
 * Obtener productos sin stock
 */
@GetMapping("/out-of-stock")
public ResponseEntity<List<Product>> getOutOfStockProducts() {
    List<Product> products = stockService.getOutOfStockProducts();
    return ResponseEntity.ok(products);
}

/**
 * Dashboard de stock
 */
@GetMapping("/dashboard")
public ResponseEntity<Map<String, Object>> getStockDashboard() {
    Map<String, Object> dashboard = stockService.getStockDashboard();
    return ResponseEntity.ok(dashboard);
}

/**
 * Verificar disponibilidad de stock
 */
@GetMapping("/check/{productId}")
public ResponseEntity<Map<String, Object>> checkStock(
    @PathVariable Long productId,
    @RequestParam Integer quantity) {

    Map<String, Object> response = new HashMap<>();
    boolean available = stockService.hasEnoughStock(productId, quantity);

    response.put("available", available);
    response.put("productId", productId);
    response.put("requestedQuantity", quantity);

    return ResponseEntity.ok(response);
}

```



```
}  
}
```

PASO 6: Actualizar InvoiceService para descontar stock

```
// ... existing code ...
```

```
@Service
```

```
public class InvoiceService {
```

```
    @Autowired
```

```
    private InvoiceRepository invoiceRepository;
```

```
    @Autowired
```

```
    private StockService stockService; // AÑADIR ESTA LÍNEA
```

```
// ... existing code ...
```

```
@Transactional
```

```
public Invoice createInvoice(Invoice invoice) {
```

```
    // Validar stock antes de crear la factura
```

```
    for (InvoiceItem item : invoice.getItems()) {
```

```
        if (!stockService.hasEnoughStock(item.getProduct().getId(), item.getQuantity())) {  
            throw new RuntimeException("Stock insuficiente para el producto: " +  
                item.getProduct().getName());  
        }  
    }
```

```
}
```

```
// Guardar factura
```

```
Invoice savedInvoice = invoiceRepository.save(invoice);
```

```
// Descontar stock de cada producto
```

```
for (InvoiceItem item : savedInvoice.getItems()) {
```

```
    stockService.reduceStock(  
        item.getProduct().getId(),  
        item.getQuantity(),  
        "INVOICE",  
        savedInvoice.getId(),  
        "System" // o el usuario actual  
    );
```

```
};  
}
```

```
    return savedInvoice;
  }

  // ... existing code ...
}
```

PASO 7: Frontend - Componentes React

1. Servicio de API para Stock

```
import axios from 'axios';

const API_URL = 'http://localhost:8080/api/v1/stock';

export const stockService = {
  // Aumentar stock
  increaseStock: async (productId, quantity, reason, userName = 'Admin') => {
    const response = await axios.post(
      `${API_URL}/increase/${productId}`,
      null,
      {
        params: { quantity, reason, userName }
      }
    );
    return response.data;
  },

  // Ajustar stock
  adjustStock: async (productId, newStock, reason, userName = 'Admin') => {
    const response = await axios.put(
      `${API_URL}/adjust/${productId}`,
      null,
      {
        params: { newStock, reason, userName }
      }
    );
    return response.data;
  },

  // Obtener movimientos de un producto
  getProductMovements: async (productId) => {
    const response = await axios.get(`${API_URL}/movements/${productId}`);
    return response.data;
  }
}
```

```

    },

    // Movimientos recientes
    getRecentMovements: async () => {
      const response = await axios.get(`${API_URL}/movements/recent`);
      return response.data;
    },

    // Productos con stock bajo
    getLowStockProducts: async () => {
      const response = await axios.get(`${API_URL}/low-stock`);
      return response.data;
    },

    // Productos sin stock
    getOutOfStockProducts: async () => {
      const response = await axios.get(`${API_URL}/out-of-stock`);
      return response.data;
    },

    // Dashboard de stock
    getStockDashboard: async () => {
      const response = await axios.get(`${API_URL}/dashboard`);
      return response.data;
    },

    // Verificar disponibilidad
    checkStock: async (productId, quantity) => {
      const response = await axios.get(`${API_URL}/check/${productId}`, {
        params: { quantity }
      });
      return response.data;
    }
  }
};

```

2. Dashboard de Stock

```

import React, { useState, useEffect } from 'react';
import { stockService } from '../services/stockService';
import toast from 'react-hot-toast';

const StockDashboard = () => {
  const [dashboard, setDashboard] = useState({
    totalProducts: 0,

```

```

    inStock: 0,
    lowStock: 0,
    outOfStock: 0,
    lowStockProducts: [],
    outOfStockProducts: [],
    recentMovements: []
  });
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    loadDashboard();
  }, []);

  const loadDashboard = async () => {
    try {
      setLoading(true);
      const data = await stockService.getStockDashboard();
      setDashboard(data);
    } catch (error) {
      toast.error('Error al cargar dashboard de stock');
      console.error(error);
    } finally {
      setLoading(false);
    }
  };

  const getStockBadge = (status) => {
    const badges = {
      IN_STOCK: 'bg-success',
      LOW_STOCK: 'bg-warning',
      OUT_OF_STOCK: 'bg-danger'
    };
    return badges[status] || 'bg-secondary';
  };

  const getMovementIcon = (type) => {
    const icons = {
      IN: 'bi-arrow-down-circle text-success',
      OUT: 'bi-arrow-up-circle text-danger',
      ADJUSTMENT: 'bi-gear text-primary'
    };
    return icons[type] || 'bi-circle';
  };

```

```

if (loading) {
  return (
    <div className="text-center p-5">
      <div className="spinner-border" role="status">
        <span className="visually-hidden">Cargando...</span>
      </div>
    </div>
  );
}

```

```

return (
  <div className="container-fluid">
    <div className="row mb-4">
      <div className="col">
        <h2>
          <i className="bi bi-box-seam me-2"></i>
          Control de Stock
        </h2>
      </div>
    </div>

    { /* Tarjetas de resumen */ }
    <div className="row mb-4">
      <div className="col-md-3">
        <div className="card bg-primary text-white">
          <div className="card-body">
            <h5 className="card-title">Total Productos</h5>
            <h2>{dashboard.totalProducts}</h2>
          </div>
        </div>
      </div>
      <div className="col-md-3">
        <div className="card bg-success text-white">
          <div className="card-body">
            <h5 className="card-title">En Stock</h5>
            <h2>{dashboard.inStock}</h2>
          </div>
        </div>
      </div>
      <div className="col-md-3">
        <div className="card bg-warning text-white">
          <div className="card-body">
            <h5 className="card-title">Stock Bajo</h5>
            <h2>{dashboard.lowStock}</h2>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        </div>
    </div>
</div>
<div className="col-md-3">
    <div className="card bg-danger text-white">
        <div className="card-body">
            <h5 className="card-title">Sin Stock</h5>
            <h2>{dashboard.outOfStock}</h2>
        </div>
    </div>
</div>
</div>
</div>

<div className="row">
    { /* Productos con stock bajo */ }
    <div className="col-md-6 mb-4">
        <div className="card">
            <div className="card-header bg-warning text-white">
                <h5 className="mb-0">
                    <i className="bi bi-exclamation-triangle me-2"></i>
                    Productos con Stock Bajo
                </h5>
            </div>
            <div className="card-body" style={{ maxHeight: '400px', overflowY: 'auto' }}>
                {dashboard.lowStockProducts.length === 0 ? (
                    <p className="text-muted">No hay productos con stock bajo</p>
                ) : (
                    <div className="list-group">
                        {dashboard.lowStockProducts.map(product => (
                            <div key={product.id} className="list-group-item">
                                <div className="d-flex justify-content-between align-items-center">
                                    <div>
                                        <h6 className="mb-1">{product.name}</h6>
                                        <small className="text-muted">
                                            Código: {product.code}
                                        </small>
                                    </div>
                                </div>
                                <div className="text-end">
                                    <span className="badge bg-warning text-dark fs-6">
                                        {product.stock} unidades
                                    </span>
                                    <br />
                                    <small className="text-muted">
                                        Mínimo: {product.minStock}
                                    </small>
                                </div>
                            </div>
                        )

```

```
</small>
</div>
</div>
</div>
)))}
</div>
)}
</div>
</div>
</div>

{ /* Productos sin stock */
<div className="col-md-6 mb-4">
  <div className="card">
    <div className="card-header bg-danger text-white">
      <h5 className="mb-0">
        <i className="bi bi-x-circle me-2"></i>
        Productos Sin Stock
      </h5>
    </div>
    <div className="card-body" style={{ maxHeight: '400px', overflowY: 'auto' }}>
      {dashboard.outOfStockProducts.length === 0 ? (
        <p className="text-muted">No hay productos sin stock</p>
      ) : (
        <div className="list-group">
          {dashboard.outOfStockProducts.map(product => (
            <div key={product.id} className="list-group-item">
              <div className="d-flex justify-content-between align-items-center">
                <div>
                  <h6 className="mb-1">{product.name}</h6>
                  <small className="text-muted">
                    Código: {product.code}
                  </small>
                </div>
                <span className="badge bg-danger fs-6">
                  Agotado
                </span>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  )}
</div>
)}
</div>
</div>
```

```

    </div>
</div>

{/* Movimientos recientes */}
<div className="row">
  <div className="col-12">
    <div className="card">
      <div className="card-header">
        <h5 className="mb-0">
          <i className="bi bi-clock-history me-2"></i>
          Movimientos Recientes
        </h5>
      </div>
      <div className="card-body">
        <div className="table-responsive">
          <table className="table table-hover">
            <thead>
              <tr>
                <th>Fecha</th>
                <th>Producto</th>
                <th>Tipo</th>
                <th>Cantidad</th>
                <th>Stock Anterior</th>
                <th>Stock Nuevo</th>
                <th>Usuario</th>
                <th>Motivo</th>
              </tr>
            </thead>
            <tbody>
              {dashboard.recentMovements.map(movement => (
                <tr key={movement.id}>
                  <td>
                    {new Date(movement.createdAt).toLocaleString()}
                  </td>
                  <td>{movement.product?.name || 'N/A'}</td>
                  <td>
                    <i className={`bi ${getMovementIcon(movement.movementType)}`}>
                      {movement.movementType}
                    </td>
                  <td>
                    <strong>{movement.quantity}</strong>
                  </td>
                  <td>{movement.previousStock}</td>
                </tr>
              )
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>
</div>

```



```

        <td>{movement.newStock}</td>
        <td>{movement.createdBy}</td>
        <td>
          <small>{movement.reason}</small>
        </td>
      </tr>
    )}
  </tbody>
</table>
</div>
</div>
</div>
</div>
</div>
</div>
);
};

```

```
export default StockDashboard;
```

3. Modal de Ajuste de Stock

```

import React, { useState } from 'react';
import { stockService } from '../services/stockService';
import toast from 'react-hot-toast';

const StockAdjustmentModal = ({ product, onClose, onSuccess }) => {
  const [adjustmentData, setAdjustmentData] = useState({
    type: 'increase', // 'increase' o 'adjust'
    quantity: '',
    newStock: product?.stock || 0,
    reason: ''
  });
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!adjustmentData.reason.trim()) {
      toast.error('Debes especificar un motivo');
      return;
    }

    try {

```

```

    setLoading(true);

    if (adjustmentData.type === 'increase') {
      if (!adjustmentData.quantity || adjustmentData.quantity <= 0) {
        toast.error('Ingresa una cantidad válida');
        return;
      }
      await stockService.increaseStock(
        product.id,
        parseInt(adjustmentData.quantity),
        adjustmentData.reason
      );
      toast.success('Stock aumentado correctamente');
    } else {
      await stockService.adjustStock(
        product.id,
        parseInt(adjustmentData.newStock),
        adjustmentData.reason
      );
      toast.success('Stock ajustado correctamente');
    }
  }

  onSuccess && onSuccess();
  onClose();
} catch (error) {
  toast.error('Error al ajustar stock');
  console.error(error);
} finally {
  setLoading(false);
}
};

return (
  <div className="modal show d-block" style={{ backgroundColor: 'rgba(0,0,0,0.5)' }}>
    <div className="modal-dialog">
      <div className="modal-content">
        <div className="modal-header">
          <h5 className="modal-title">
            Ajustar Stock - {product?.name}
          </h5>
          <button
            type="button"
            className="btn-close"
            onClick={onClose}

```

```

        disabled={loading}
      ></button>
    </div>
    <form onSubmit={handleSubmit}>
      <div className="modal-body">
        <div className="alert alert-info">
          <strong>Stock Actual:</strong> {product?.stock} unidades
        </div>

        <div className="mb-3">
          <label className="form-label">Tipo de Ajuste</label>
          <select
            className="form-select"
            value={adjustmentData.type}
            onChange={(e) => setAdjustmentData({
              ...adjustmentData,
              type: e.target.value
            })}
            disabled={loading}
          >
            <option value="increase">Aumentar Stock</option>
            <option value="adjust">Ajuste Manual</option>
          </select>
        </div>

        {adjustmentData.type === 'increase' ? (
          <div className="mb-3">
            <label className="form-label">Cantidad a Agregar</label>
            <input
              type="number"
              className="form-control"
              value={adjustmentData.quantity}
              onChange={(e) => setAdjustmentData({
                ...adjustmentData,
                quantity: e.target.value
              })}
              min="1"
              required
              disabled={loading}
            />
            <small className="text-muted">
              Nuevo stock: {product?.stock + parseInt(adjustmentData.quantity || 0)}
            </small>
          </div>
        ) : null}
      </div>
    </form>
  )}

```

```

): (
  <div className="mb-3">
    <label className="form-label">Nuevo Stock</label>
    <input
      type="number"
      className="form-control"
      value={adjustmentData.newStock}
      onChange={(e) => setAdjustmentData({
        ...adjustmentData,
        newStock: e.target.value
      })}
      min="0"
      required
      disabled={loading}
    />
  </div>
)}

<div className="mb-3">
  <label className="form-label">Motivo *</label>
  <textarea
    className="form-control"
    rows="3"
    value={adjustmentData.reason}
    onChange={(e) => setAdjustmentData({
      ...adjustmentData,
      reason: e.target.value
    })}
    required
    disabled={loading}
    placeholder="Ej: Entrada de mercancía, corrección de inventario..."
  ></textarea>
</div>
</div>
<div className="modal-footer">
  <button
    type="button"
    className="btn btn-secondary"
    onClick={onClose}
    disabled={loading}
  >
    Cancelar
  </button>
  <button

```

```

        type="submit"
        className="btn btn-primary"
        disabled={loading}
      >
      {loading ? (
        <>
          <span className="spinner-border spinner-border-sm me-2"></span>
          Guardando...
        </>
      ) : (
        'Guardar Ajuste'
      )}
    </button>
  </div>
</form>
</div>
</div>
</div>
);
};

```

export default StockAdjustmentModal;

4. Indicador de Stock en Productos

```

import React from 'react';

const StockBadge = ({ stock, minStock, stockStatus }) => {
  const getBadgeClass = () => {
    switch (stockStatus) {
      case 'OUT_OF_STOCK':
        return 'bg-danger';
      case 'LOW_STOCK':
        return 'bg-warning text-dark';
      case 'IN_STOCK':
        return 'bg-success';
      default:
        return 'bg-secondary';
    }
  };

  const getIcon = () => {
    switch (stockStatus) {
      case 'OUT_OF_STOCK':

```

```

        return 'bi-x-circle';
    case 'LOW_STOCK':
        return 'bi-exclamation-triangle';
    case 'IN_STOCK':
        return 'bi-check-circle';
    default:
        return 'bi-circle';
    }
};

const getStatusText = () => {
    switch (stockStatus) {
        case 'OUT_OF_STOCK':
            return 'Sin Stock';
        case 'LOW_STOCK':
            return 'Stock Bajo';
        case 'IN_STOCK':
            return 'En Stock';
        default:
            return 'Desconocido';
    }
};

return (
    <div className="d-flex align-items-center">
        <span className={`badge ${getBadgeClass()} me-2`} >
            <i className={`bi ${getIcon()} me-1`} ></i>
            {stock} unidades
        </span>
        <small className="text-muted">
            {getStatusText()}
        </small>
    </div>
);
};

export default StockBadge;

```

PASO 8: Integrar en el Formulario de Productos

Actualiza tu formulario de productos para incluir los campos de stock:

```
// En tu componente de ProductForm.jsx
<div className="mb-3">
  <label className="form-label">Stock Inicial</label>
  <input
    type="number"
    className="form-control"
    value={productData.stock}
    onChange={(e) => setProductData({...productData, stock: e.target.value})}
    min="0"
    required
  />
</div>

<div className="mb-3">
  <label className="form-label">Stock Mínimo (Alerta)</label>
  <input
    type="number"
    className="form-control"
    value={productData.minStock}
    onChange={(e) => setProductData({...productData, minStock: e.target.value})}
    min="1"
    required
  />
  <small className="text-muted">
    Se mostrará una alerta cuando el stock llegue a este nivel
  </small>
</div>
```

Cómo Usar el Sistema

1. Configurar Base de Datos

```
mysql -u root -p < billing_app.sql
```

2. Ejecutar Backend

```
cd billingsoftware
./mvnw spring-boot:run
```

3. Ejecutar Frontend

```
cd client
npm install
npm run dev
```

4. Rutas a Agregar en tu Router

```
import StockDashboard from './components/stock/StockDashboard';
```

```
// En tu App.jsx o Router
<Route path="/stock" element={<StockDashboard />} />
```

🌟 Características Implementadas

✅ **Control automático de stock** al generar facturas ✅ **Alertas de stock bajo** con indicadores visuales ✅ **Historial completo** de movimientos ✅ **Ajustes manuales** de inventario con motivos ✅ **Dashboard en tiempo real** con estadísticas ✅ **Validación de stock** antes de ventas ✅ **Indicadores de estado** (En Stock, Stock Bajo, Sin Stock) ✅ **Auditoría completa** de cambios

¿Quieres que te ayude con alguna parte específica o agregar más funcionalidades como notificaciones push, exportación de reportes o integración con códigos de barras? 📦